COURSE OUTLINE

Gennoor Tech
TRAIN. INNOVATE. BUILD.

Agentic Configuration Reviewer

Review infrastructure and application configuration before deployment

2 days (8 hours per day) | Intermediate | Instructor-led, hands-on labs | Capstone project

About this course

Who should attend

DevSecOps, platform and cloud engineers, infrastructure-as-code authors, security champions in engineering teams.

Prerequisites

- Terraform or Kubernetes YAML basics
- Git and pull-request workflow
- Python fundamentals
- Policy-as-code concepts are helpful

What you will be able to do

- Parse and normalize Terraform plan output and Kubernetes YAML while preserving file, line and resource provenance
- Combine deterministic policy rules with AI-assisted context and explanation
- Generate minimal remediation diffs and validate them with native tooling
- Integrate review results into a pull request as read-only comments that never auto-merge
- Build a rule test suite and measure precision, missed cases and noisy alerts

Tools and technologies

Claude | Terraform | Kubernetes | YAML | JSON | Python | GitHub | GitHub Actions | pytest | Trivy | Snyk

Table of contents

For more info - https://gennoor.com

DAY 1 3 modules

MODULE 1

Supported formats, parsers and provenance

- Start with Terraform JSON plan output and Kubernetes YAML; no promise of universal understanding
- Native parsers and established libraries; file, line, resource ID and source context on every finding
- Malformed-input tests

HANDS-ON LAB
Implement the parser adapters and prove provenance survives normalization.

DELIVERABLE
Parsers with provenance.

MODULE 2

Deterministic policy rules

- A small ruleset for clearly testable issues: public exposure, privileged workloads, overly broad permissions
- Rules with stable IDs, severity, rationale, references, exceptions and test cases
- The LLM is never the authority on whether a rule is violated

HANDS-ON LAB
Write the rules and their fixtures, including secure configurations to measure false positives.

DELIVERABLE
Deterministic configuration scanner.

MODULE 3

AI explanations in a strict schema

- The model receives the parsed resource and rule evidence
- Concise explanation, uncertainty and remediation rationale in a strict schema
- Structured outputs, evidence grounding, uncertainty handling

HANDS-ON LAB
Add the explanation agent and verify it cannot change a policy outcome.

DELIVERABLE
AI-augmented review report.

DAY 2 3 modules

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

MODULE 4

Candidate diffs and native validation

- Minimal patches only for supported cases; no rewriting unrelated files
- Formatting, syntax validation, native tooling, policy checks and tests on the modified configuration
- Regression tests and exception handling

HANDS-ON LAB
Generate and validate a fix for each finding; watch an invalid patch get rejected.

DELIVERABLE
Validated patch workflow.

MODULE 5

Pull-request integration, read-only by default

- Local review report first, then a mock pull-request comment
- Real integrations only with a sandbox account and explicit authorization; never auto-merge
- Safe handling of untrusted repository content; audit trail

HANDS-ON LAB
Wire the reviewer into a sandbox repository's pull requests with a CI workflow.

DELIVERABLE
CI-integrated reviewer.

MODULE 6

Evaluation, developer experience and capstone

- Precision and recall on the fixture set, unsupported statements, patch validity, review noise
- Developer UX and productization
- Capstone demo and limitations

HANDS-ON LAB
Run the evaluation, tune noise, and present the working configuration reviewer.

DELIVERABLE
Working configuration reviewer.

Capstone

Gennoor Tech
INNOVATE. BUILD.
For more info https://gennoor.com

Capstone project

A developer workflow that parses supported configuration files, runs deterministic policy checks, uses AI for context and remediation drafts, validates proposed changes natively, and comments on a pull request without ever merging.

Assessment

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

Engineering principles used throughout

- Model output is a proposal, not a fact: every finding and generated change is validated by deterministic checks and tests.
- Authorization lives outside the model: tool access is enforced by application code, scoped credentials and policy.
- Human approval for consequential actions: deploying a mitigation, changing production configuration, rotating credentials.
- Decisions, evidence, model and tool versions, approvals and outcomes are logged without recording secrets.
- A model adapter and a tool adapter layer keep every build provider-agnostic and enterprise-portable.
- Quality is measured: false positives, false negatives, reliability, latency, cost and safe failure, not just a demo.

© Gennoor Tech · Train. Innovate. Build.